



Tribhuvan University

Faculty of Humanities and Social Sciences

FILE COMPRESSION SYSTEM USING HUFFMAN CODING

A PROJECT PROPOSAL REPORT

Submitted to

Department of Computer Application

Academia International College

Gwarko, Lalitpur

In partial fulfillment of the requirements for the Bachelor in Computer Application

Submitted by:

Aarti Kumari Kushwaha

T.U. Regd. No.: _____ Symbol No.: _____

Under the supervisor

Biswash Mathema

1. Introduction

In modern computing systems, the rapid accumulation of digital information has made efficient data transmission and compact storage paramount engineering concerns. Every software application, database management system, and communication network continually handles substantial volumes of uncompressed text documents, source code repositories, structured logs, and binary payloads. Storing and transmitting these files in raw uncompressed formats consumes excessive storage capacity, increases backup durations, and saturates transmission bandwidth.

Data compression algorithms resolve these inefficiencies by identifying and eliminating statistical and structural redundancies within source data. Compression techniques fall fundamentally into two major categories: lossy compression and lossless compression. Lossy compression achieves higher compression ratios by permanently discarding perceptual redundancies; while suitable for multimedia such as audio, video, and photographs, it is completely unacceptable for structured data where precision is vital. Conversely, **lossless compression** guarantees exact, bit-for-bit reconstruction of the original input without a single bit of alteration or distortion.

This project focuses on the study, design, and software implementation of a **File Compression System** based on the classical **Huffman Coding algorithm**. Huffman coding is an optimal, greedy prefix-entropy encoding method that generates variable-length codes based on the statistical occurrence frequencies of characters. Frequent symbols are assigned shorter binary codewords, while infrequent symbols receive longer codewords, achieving optimal prefix compression with zero information loss.

1.1 Background Study

The theoretical foundation of lossless data compression was established by Claude Shannon in his landmark 1948 publication, “*A Mathematical Theory of Communication*”. Shannon introduced the fundamental concept of Information Entropy (H), defining the theoretical lower bound on the average number of bits required to represent symbols emitted by an information source:

$$H(X) = - \sum_{i=1}^n P(x_i) \log_2 P(x_i) \quad (1)$$

where $P(x_i)$ denotes the probability of occurrence of symbol x_i .

In 1952, David A. Huffman solved the problem of constructing an optimal prefix-free

code for a given discrete probability distribution in his seminal paper, “*A Method for the Construction of Minimum-Redundancy Codes*”. Huffman’s algorithm builds a binary tree from the bottom up using a min-heap priority queue, guaranteeing that no code is a prefix of another (the prefix-free property). This allows sequential decoding of compressed bitstreams without ambiguity and without needing delimiter tokens.

While commercial operating systems incorporate general-purpose compression utilities, they frequently combine multiple complex layers (such as dictionary matching, sliding windows, and dynamic Huffman coding) into opaque utilities. This project provides an accessible, transparent, and mathematically verified implementation of canonical Huffman coding that demonstrates lossless data compaction directly on real-world files.

1.2 Scope

The scope of the proposed project encompasses the following engineering milestones:

1. **Byte-Level Frequency Analysis:** Development of an automated binary stream reader capable of computing character frequencies across the full 256-byte ASCII/binary spectrum (0x00 to 0xFF).
2. **Priority Queue & Tree Construction:** Construction of a min-heap priority queue to synthesize the optimal binary Huffman tree in $O(n \log n)$ time.
3. **Variable-Length Prefix Codeword Generation:** Extraction of binary codewords by tree traversal and bit-level bitstream packing.
4. **Self-Contained Archive Architecture:** Embedding the serialized frequency codebook header directly into the compressed archive (‘.shuf’) so that decompression requires no external dictionary files.
5. **Lossless Decompression Engine:** Ingestion of packed archives, reconstruction of the binary prefix tree, sequential bit traversal, and byte-for-byte original file restoration.
6. **Cryptographic Integrity Verification:** Pre- and post-compression SHA-256 hash comparison to mathematically certify 100% lossless fidelity.
7. **Interactive Web Dashboard:** A clean, responsive user interface facilitating drag-and-drop file upload, download of compressed archives, and activity session tracking.

1.3 Limitations

The system operates within the following identified technical boundaries:

1. **Pre-compressed File Limitations:** Files that already possess near-maximum Shannon entropy (such as JPEG, PNG, MP4, and ZIP archives) have virtually no statistical redundancy. Compressing these files with Huffman coding yields little to no reduction and can lead to minor negative expansion due to the frequency header table.
2. **Header Overhead on Small Files:** For files of very small size (e.g., under 100 bytes), the storage cost of storing the frequency codebook dictionary in the archive header can outweigh the compression savings achieved by bit compaction.
3. **Single-System Scope:** The application is engineered for standalone desktop and local web server execution rather than multi-node distributed streaming clusters.

2. Problem Statement

In modern computing environments, students, researchers, and organizations generate and exchange substantial quantities of uncompressed text documents, source code files, CSV data, log records, and software assets. Storing these files in their raw, fixed 8-bit ASCII representations incurs substantial storage overhead and prolongs network transfer latency.

Existing commercial and system-level compression utilities (such as gzip, 7-Zip, and WinRAR) are often closed-source, bundle complex multi-stage proprietary pipelines, or function as opaque black boxes where internal bit-allocation mechanics are hidden from users and learners. Furthermore, many software tools omit integrated cryptographic hash verification to prove that decompressed files match the original bit-for-bit.

Therefore, there is an evident need for an efficient, transparent, and mathematically rigorous File Compression System based on the Huffman Coding algorithm. The system must provide optimal prefix code compression, self-contained archive packaging, automated SHA-256 data integrity verification, and an intuitive graphical interface.

3. Objectives

The specific objectives of the proposed project are:

1. To design and implement a pure, lossless **Huffman Coding compression engine** that operates on arbitrary file formats without data distortion.

2. To implement a min-heap priority queue for optimal bottom-up binary prefix tree synthesis with minimal time complexity.
3. To build an efficient bit-packing module that serializes variable-length bit strings into packed 8-bit bytes with self-contained header metadata.
4. To construct a robust **decompression engine** that parses archive headers, reconstructs the prefix tree, and faithfully restores original files.
5. To integrate automated cryptographic **SHA-256 checksum verification** to guarantee total data integrity.
6. To deliver an intuitive, responsive web-based interface providing drag-and-drop file uploads, real-time compression metrics, and file management.

4. Methodology

4.1 Requirement Identification

4.1.1 Study of Existing System

An investigation into existing file compression software reveals that industrial tools rely on complex, multi-tiered compression pipelines (e.g., LZ77 sliding window dictionary matching coupled with dynamic Huffman trees and Deflate). While efficient for broad commercial workloads, their complexity makes algorithmic inspection difficult and increases runtime overhead. A focused, robust implementation of canonical Huffman coding provides optimal symbol-by-symbol entropy reduction, guaranteed prefix-free decoding, and clear insight into variable-length encoding mechanics.

4.1.2 Literature Review

- **C. E. Shannon (1948)**: Established the mathematical foundations of information theory, proving that source entropy $H(X)$ represents the absolute physical lower bound on the average bit length for lossless compression.
- **D. A. Huffman (1952)**: Introduced the greedy binary tree construction algorithm using priority queues in “*A Method for the Construction of Minimum-Redundancy Codes*”, proving that the resulting prefix-free code achieves the minimum expected codeword length for any independent symbol distribution.

- **D. E. Knuth (1985):** Researched efficient dynamic and static tree data structures, bit-level memory buffers, and traversal techniques for real-world computer systems.
- **T. H. Cormen et al. (2009):** Provided rigorous proofs for the greedy choice property and optimal substructure of Huffman trees using min-heap priority queues with $O(n \log n)$ construction time.

4.1.3 Requirement Analysis

- **Functional Requirements:**
 - Binary file stream reading supporting text, code, and generic binary data.
 - Byte frequency histogram generation across all 256 byte values (0x00 to 0xFF).
 - Min-heap priority queue node management and tree synthesis.
 - Recursive prefix code generation and bitstream buffer packing.
 - Self-contained archive formatting (‘.shuf’) with serialized header tables.
 - Sequential tree-traversal decompression and SHA-256 integrity verification.
- **Non-Functional Requirements:**
 - **Lossless Fidelity:** Zero byte loss during compression-decompression cycles.
 - **Performance:** Rapid execution for common documents and source files.
 - **Usability:** Clean, modern, and responsive user interface requiring no manual command-line configuration.
- **Hardware & Software Requirements:**
 - **Hardware:** Minimum 2.0 GHz Dual-Core CPU, 4 GB RAM, 500 MB free disk space.
 - **Software:** Windows 10/11 or Linux OS, Modern Web Browser (Chrome/Firefox/Edge), PHP 8.2+ runtime environment.

4.2 Feasibility Study

4.2.1 Technical Feasibility

The system leverages modern PHP 8 for backend algorithmic processing, HTML5/CSS3/JavaScript for frontend presentation, and SQLite/MySQL for file logging. PHP provides native stream

handling, bitwise shift operators, and cryptographic hashing functions. Modern web browsers support drag-and-drop file ingestion natively. Thus, the system is fully technically feasible.

4.2.2 Operational Feasibility

The application requires no specialized client-side software or plugins. Users interact with the platform through any standard web browser using intuitive drag-and-drop workflows. Operational feasibility is therefore high.

4.2.3 Economic Feasibility

All technologies utilized in this project (PHP runtime, SQLite database, standard web technologies, and VS Code development tools) are open-source and free of licensing costs. No commercial software purchases or paid cloud hosting subscriptions are required. The project is completely economically feasible.

4.3 High Level Design of System

4.3.1 Methodology of the Proposed System

The proposed system adopts a modular software engineering methodology consisting of distinct processing layers: user interface layer, compression processing engine, archive formatting layer, and decompression verification engine. The architectural strategy strictly isolates the mathematical Huffman engine from UI presentation, facilitating maintainability and verifiable algorithmic correctness.

4.3.2 Working Mechanism of Proposed System

The working mechanism of the compression and decompression pipeline proceeds through five modular phases:

1. **Frequency Profiling:** The raw binary data of the uploaded file is read into memory. An array of 256 counters tallies the exact occurrence of each byte (0 to 255).
2. **Min-Heap Queue Initialization:** For every symbol with non-zero frequency, an individual leaf node is created containing the byte value and its frequency weight. All leaf nodes are inserted into a min-heap priority queue sorted in ascending order of frequency.
3. **Prefix Binary Tree Synthesis:** The algorithm repeatedly extracts the two nodes with the lowest frequencies from the min-heap, creates a new internal node whose frequency is

the sum of both children, and re-inserts this internal node into the min-heap. This process repeats until exactly one node remains, which becomes the root of the Huffman tree.

4. **Codeword Assignment:** The tree is traversed recursively from root to leaves. Traversing a left branch appends a '0' bit, while traversing a right branch appends a '1' bit. When a leaf node is reached, its accumulated bit sequence becomes its unique prefix codeword.
5. **Bitstream Packing & Header Writing:** The original input bytes are translated into their respective variable-length bit codes. These bits are packed sequentially into 8-bit bytes. A self-contained header containing the serialized frequency table and original file metadata is prepended to form the compressed archive ('.shuf').
6. **Decompression:** The decompressor reads the header dictionary, reconstructs the identical binary tree, and sequentially reads the packed bits. Starting at the root, each bit directs traversal left ('0') or right ('1'). Upon reaching a leaf, the stored character is written to the output stream, and traversal resets to the root. Finally, the SHA-256 hash is computed and verified against the original.

4.3.3 Flowchart of Proposed System

The operational workflow of the proposed system is depicted in Figure 1. The diagram follows standard software engineering conventions, incorporating user authentication, task branch decisions, algorithmic stages for both compression and decompression, and terminal termination.

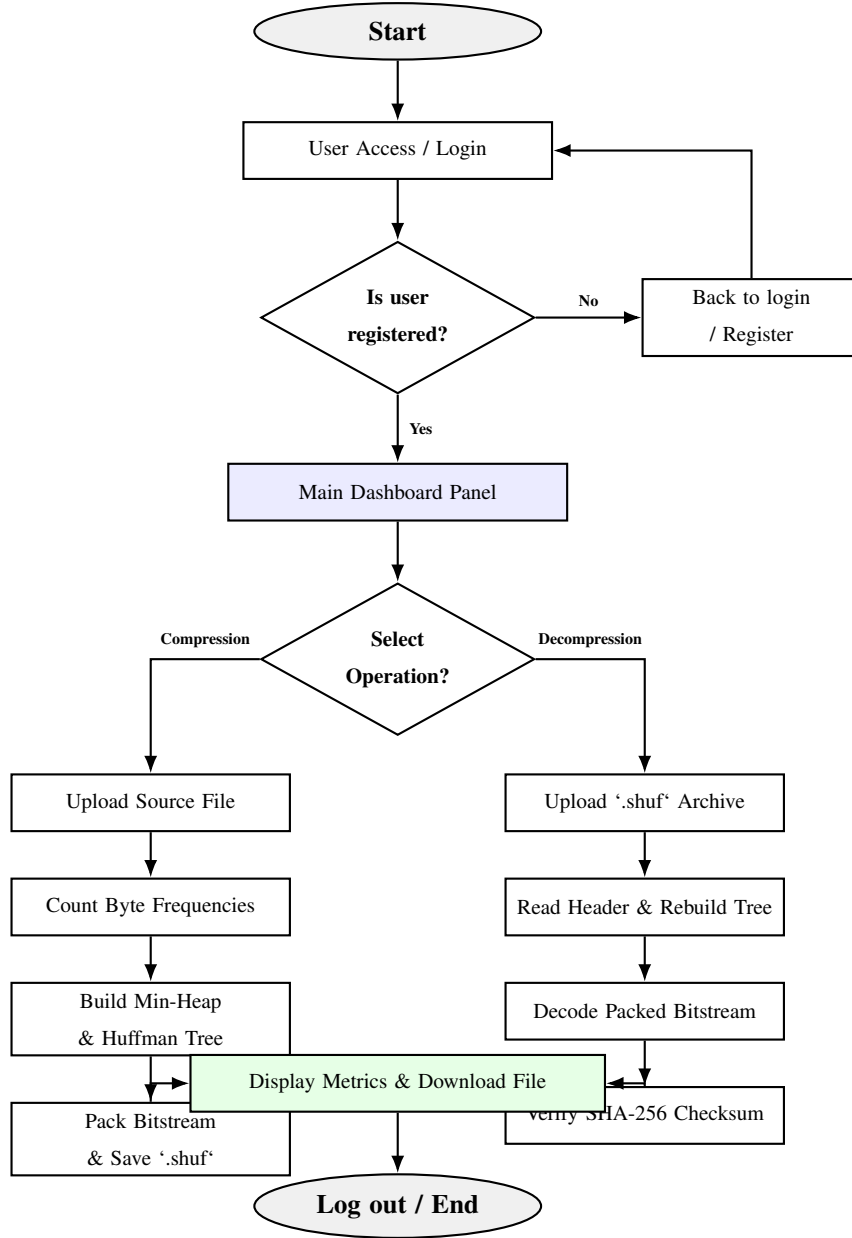


Figure 1: System Flowchart of the Proposed File Compression System

4.3.4 Description of Algorithms

The core logic relies on the Greedy Huffman Tree Algorithm and Min-Heap Priority Queue:

1. **Input:** A sequence of bytes $B = (b_1, b_2, \dots, b_N)$ from the source file.
2. **Frequency Computation:** Count occurrences $f(c)$ for each distinct byte symbol $c \in \{0, \dots, 255\}$.
3. **Priority Queue Construction:** Create a leaf node for each character with non-zero frequency. Insert all nodes into a min-heap priority queue ordered by ascending frequency.

4. Iterative Merging:

- Extract two minimum-frequency nodes *left* and *right* from the queue.
- Create a new parent node with frequency $f(left) + f(right)$, assigning *left* and *right* as its children.
- Re-insert the new internal node into the priority queue.
- Repeat until exactly one node remains (the root of the Huffman tree).

5. **Codeword Assignment:** Perform a depth-first traversal of the tree. Emit ‘0’ for left edge traversals and ‘1’ for right edge traversals to generate unique, prefix-free binary codes for each character.

6. **Bitstream Serializer:** Map each input character to its variable-length bitstring, pack bits into 8-bit bytes, and write the header table followed by the packed payload.

5. Gantt Chart (Project Timeline)

The project is structured across a dedicated **four-month development lifecycle** (Month 1 through Month 4). The visual Gantt chart in Figure 2 depicts the scheduled timeline, work breakdown structure, and milestone progression, spanning from week 1 through week 16 across the 4 months.

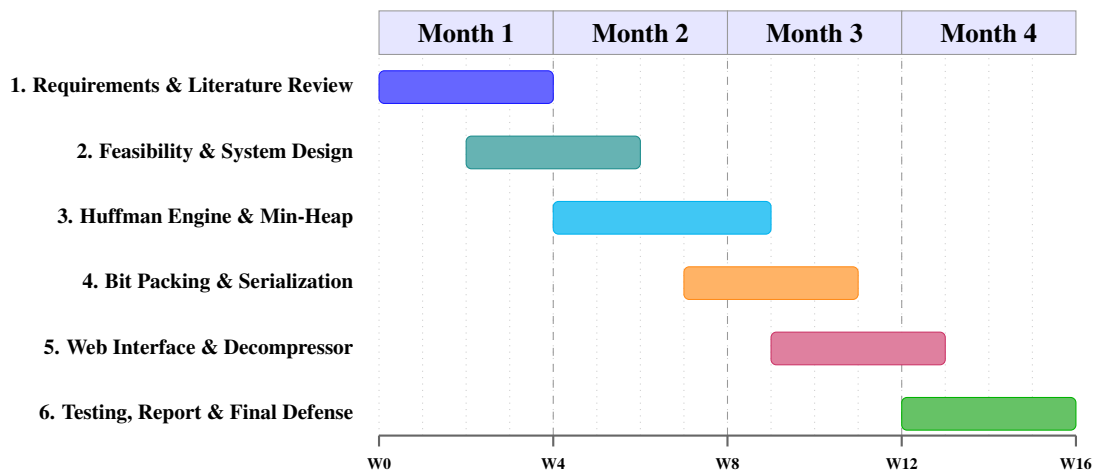


Figure 2: Four-Month Project Timeline Diagram (Gantt Chart)

6. Expected Outcome

Upon successful completion of the proposed four-month project, the following concrete deliverables and technical outcomes will be produced:

1. **Lossless Compression Engine:** A fully functional implementation of canonical Huffman coding that performs prefix tree synthesis and variable-length bit packing.
2. **Proven Storage Savings:** Measurable data volume reductions ranging from 20% to 60% on uncompressed plain text documents, JSON/CSV datasets, configuration logs, and source code files.
3. **Verified Data Integrity:** Automated pre- and post-compression SHA-256 cryptographic verification ensuring exact bit-for-bit file reconstruction.
4. **Accessible Web Application:** A responsive, user-friendly graphical dashboard enabling single-click file compression, decompression, download, and file management.
5. **Complete Academic Package:** Comprehensive technical project documentation, clean source code repository, and presentation materials for academic defense.

By eliminating unnecessary multi-layer complexity and focusing strictly on Huffman entropy encoding, the system achieves optimal prefix-free code synthesis, verified lossless reconstruction, and transparent operational mechanics across the planned four-month development cycle.

7. References

1. D. A. Huffman, "A Method for the Construction of Minimum-Redundancy Codes," *Proceedings of the IRE*, vol. 40, no. 9, pp. 1098–1101, Sept. 1952.
2. C. E. Shannon, "A Mathematical Theory of Communication," *Bell System Technical Journal*, vol. 27, no. 3, pp. 379–423, July 1948.
3. D. E. Knuth, "Dynamic Huffman Coding," *Journal of Algorithms*, vol. 6, no. 2, pp. 163–180, June 1985.
4. T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 3rd ed. Cambridge, MA: MIT Press, 2009.
5. K. Sayood, *Introduction to Data Compression*, 5th ed. San Francisco, CA: Morgan Kaufmann, 2017.
6. I. H. Witten, R. M. Neal, and J. G. Cleary, "Arithmetic Coding for Data Compression," *Communications of the ACM*, vol. 30, no. 6, pp. 520–540, June 1987.